

Relazione per “Programmazione ad Oggetti”

Fortarezza Gabriele, Franzolini Alice, Magi Sabrina, Lorenzo Vasini

15 GIUGNO 2026

Indice

1 Analisi

1.1 Descrizione e requisiti	2
1.2 Modello del Dominio	3

2 Design

2.1 Architettura	5
2.2 Design dettagliato	8

3 Sviluppo

3.1 Testing automatizzato	
3.2 Metodologia di lavoro	
3.2 Note di sviluppo	

4 Commenti finali

4.1 Autovalutazione e lavori futuri	
4.2 Difficoltà incontrate e commenti per i docenti	

A Guida utente

Capitolo 1 — Analisi

1.1 Descrizione e requisiti

Il progetto consiste in un videogioco di tipo platformer 2D ispirato al classico Super Mario Bros. Il giocatore controlla un protagonista che deve percorrere livelli a scorrimento orizzontale, superare ostacoli, sconfiggere nemici e raccogliere oggetti, con l'obiettivo di raggiungere la fine di ogni livello. Il gioco supporta tre livelli distinti, selezionabili dal menu iniziale e costruiti dinamicamente attraverso classi di definizione dedicate.

Requisiti funzionali

- Il protagonista può muoversi all'interno del livello spostandosi a destra e a sinistra, saltando verso l'alto o cadendo per effetto della gravità.
- Il protagonista può trovarsi in tre stati distinti: SMALL (piccolo, stato iniziale), BIG (grande, dopo aver raccolto un fungo) e INVINCIBLE (invincibile, dopo aver raccolto una stella).
- Il protagonista può raccogliere potenziamenti (power-up) che ne modificano le capacità:
 - Fungo (Mushroom): trasforma il protagonista da SMALL a BIG, aumentandone l'altezza e la resistenza ai danni.
 - Stella (Star): rende il protagonista temporaneamente invincibile, permettendogli di eliminare i nemici al semplice contatto.
- Il protagonista può subire danni dai nemici:
 - Se è BIG, torna allo stato SMALL perdendo il potenziamento attivo.
 - Se è SMALL e viene colpito da un nemico pericoloso, muore e perde una vita.
 - Cadere in un abisso causa la morte immediata.
- Il protagonista può incontrare due tipi di nemici:
 - Goomba: si muove orizzontalmente e può essere eliminato saltando sulla sua testa, entrando nello stato SQUISHED prima di scomparire.
 - Koopa: si muove orizzontalmente. Saltandoci sopra si ritira nel guscio (stato SHELL); colpendo nuovamente il guscio esso scivola ad alta velocità (stato SHELL_MOVING), eliminando nemici sul percorso.
- Il protagonista può interagire con i blocchi presenti nel livello:
 - Blocco Normale (NORMAL): indistruttibile, usato come piattaforma.

- Blocco Mattone (BRICK): rappresenta un tipo di blocco distinto dal blocco normale e dal blocco punto interrogativo.
- Blocco Punto Interrogativo (QUESTION): può produrre una moneta o un power-up al primo colpo, poi diventa inerte.
- Il protagonista può raccogliere monete. Ogni 100 monete raccolte si ottiene una vita extra.
- Il punteggio viene incrementato raccogliendo monete; il sistema di punteggio tiene inoltre traccia del numero di monete e delle vite del giocatore.
- Il livello è completato quando il protagonista raggiunge la fine del livello (entro una distanza prefissata dal bordo destro della mappa).
- Il sistema di gioco include la gestione delle vite e uno stato di game over
- Il gioco espone un HUD (Heads-Up Display) che mostra in tempo reale il punteggio corrente, il numero di monete e le vite rimanenti.
- Il gioco presenta un menu iniziale da cui è possibile selezionare il livello da avviare. Al termine di un livello il gioco ritorna automaticamente al menu principale.
- È possibile mettere in pausa e riprendere la partita in qualsiasi momento.

Requisiti non funzionali

- L'applicazione deve essere in grado di girare su qualsiasi hardware standard supportato dalla JVM.
- L'interfaccia grafica deve ridimensionarsi in base alla risoluzione dello schermo, mantenendo la camera centrata sul giocatore.
- Non devono verificarsi errori durante il gameplay, come la compenetrazione del personaggio nei blocchi o crash durante l'esecuzione.
- Le animazioni dei personaggi devono essere fluide e reattive, gestite con una classe apposita
- La definizione dei livelli deve essere separata dalla logica comune di costruzione, in modo da consentire l'aggiunta o la modifica di un livello intervenendo principalmente sulla relativa classe di configurazione.

1.2 Analisi e modello del dominio

Il dominio applicativo è composto da entità che interagiscono in un mondo bidimensionale. Ogni entità ha una posizione e una dimensione nello spazio di gioco. Le entità si dividono in due grandi categorie: le entità statiche, che non si muovono e costituiscono la struttura del livello (blocchi), e le entità dinamiche, che si muovono e sono soggette a gravità e collisioni fisiche (giocatore, nemici, power-up, monete in movimento).

Il livello (Level) funge da contenitore di tutte le entità e coordina il ciclo di aggiornamento e rendering. Al suo interno convivono il protagonista (Player), i nemici (Enemy), i blocchi (Block), le monete (Coin) e i power-up (PowerUp).

La difficoltà principale risiede nella gestione delle collisioni tra le entità dinamiche e quelle statiche: il protagonista deve poter camminare sui blocchi senza cadere per effetto della gravità, sconfiggere i nemici con le interazioni corrette, raccogliere monete e power-up, e completare il livello raggiungendo il bordo destro della mappa.

Il sistema di punteggio (ScoreSystem) tiene traccia del punteggio, delle monete raccolte e delle vite rimanenti, aggiornandosi in risposta agli eventi di gioco. La gestione degli input è separata dalla logica di gioco: il controller degli input (InputController) traduce i comandi fisici della tastiera in azioni sul modello, senza che il modello conosca la sorgente degli input.

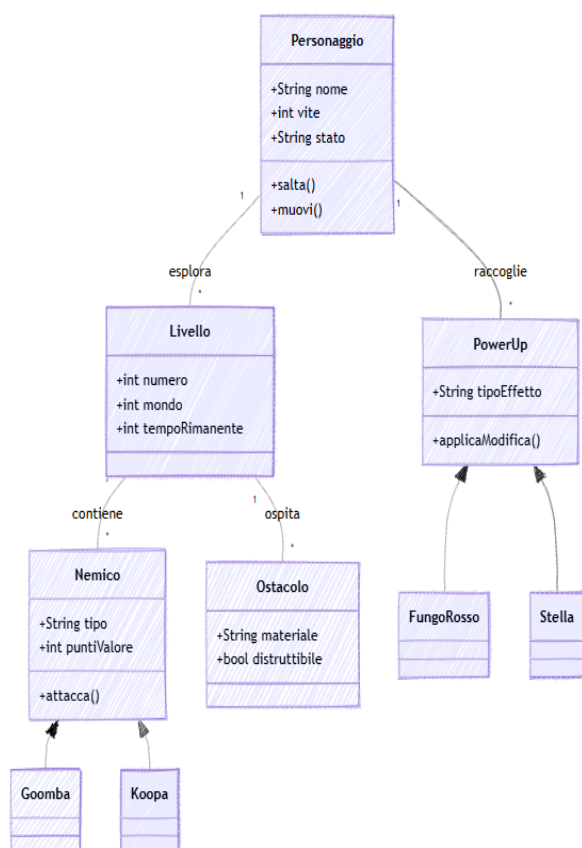


Figura 1.1: Schema UML dell'analisi del dominio, con rappresentate le entità principali ed i rapporti fra loro

Capitolo 2 — Design

2.1 Architettura

L'architettura presenta alcuni aspetti positivi. La presenza di interfacce come Level, Player e PowerUp permette di separare il contratto dalle implementazioni concrete, rendendo più chiari i ruoli principali del model e facilitando l'estensione futura del sistema. Anche la gestione degli input tramite InputController contribuisce a mantenere separata la lettura dei comandi dell'utente dalla logica interna del player: il controller interpreta gli input e li traduce in azioni di gioco, mentre il player si occupa esclusivamente del proprio comportamento.

La gerarchia delle entità rappresenta inoltre un punto importante dell'architettura, perché consente di distinguere tra oggetti statici e dinamici. In questo modo il livello può gestire in maniera uniforme le entità presenti nel mondo di gioco, mantenendo però una separazione tra oggetti fermi, come blocchi e piattaforme, e oggetti soggetti a movimento o aggiornamento fisico, come player, nemici e power-up.

Il principale limite dell'architettura riguarda la separazione non completa tra model e view. Poiché diverse entità espongono direttamente un metodo render(), la rappresentazione grafica non è completamente isolata dal model. Di conseguenza, una sostituzione completa della libreria grafica richiederebbe modifiche anche ad alcune entità, mentre in un MVC più rigoroso il model dovrebbe rimanere indipendente dalla view.

Nel progetto GameManager svolge il ruolo di coordinatore centrale tra input, livello corrente, punteggio, HUD e stato della partita. Questa scelta ha permesso di avere un unico punto di controllo per il menu iniziale, la selezione dei livelli, la pausa, il game over, la vittoria e il ritorno automatico al menu. InputController, LevelLoader e HudView rimangono componenti separati: InputController si occupa esclusivamente della lettura e del consumo degli input, LevelLoader della costruzione dei livelli, mentre HudView della visualizzazione di menu, informazioni di gioco e messaggi di stato.

Infine, in alcuni punti del progetto vengono utilizzate implementazioni concrete, come PlayerImpl, invece delle relative interfacce. Questo introduce un accoppiamento maggiore e riduce in parte la flessibilità del sistema. Un possibile miglioramento futuro consisterebbe nell'utilizzare più sistematicamente le interfacce nei punti di interazione tra i componenti principali.

2.2 Design Dettagliato

Alice Franzolini

2.2.1 Alice — Gerarchia delle entità e gestione dei nemici

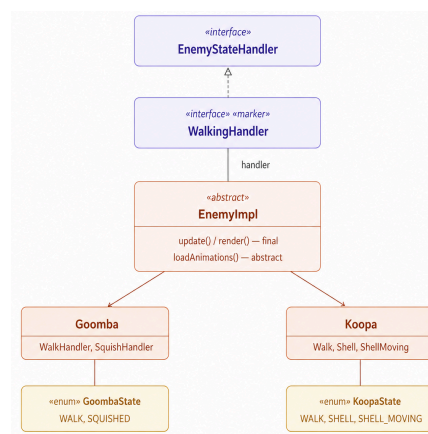
Nemici (Enemy, EnemyImpl, Goomba, Koopa)

Mi sono occupata interamente della modellazione e implementazione dei nemici del gioco, dalla definizione del contratto pubblico tramite l'interfaccia **Enemy** fino alle implementazioni concrete **Goomba** e **Koopa**.

Ho progettato la classe astratta `AbstractEnemyImpl` come base comune, centralizzando in essa la logica condivisa di animazione, fisica e dispatching degli stati tramite il campo `handler`. La scelta di una classe astratta anziché un'interfaccia standalone risponde alla necessità di riutilizzare codice condiviso tra nemici di tipo diverso, evitando duplicazioni e garantendo coerenza nel ciclo di vita di ogni entità nemica.

La gestione degli stati interni è stata resa esplicita attraverso l'uso di `Optional<EnemyStateHandler>`, evitando controlli espliciti su null e rendendo il codice più leggibile e manutenibile. La transizione tra stati avviene tramite il metodo `transitionTo()`, invocato dalle sottoclassi nei momenti opportuni: ad esempio, quando un Koopa viene calpestato entra nello stato `SHELL`, e da lì può passare a `SHELL_MOVING` se colpito nuovamente. Questa struttura realizza di fatto uno `State Pattern`, in cui ogni stato incapsula il proprio comportamento e delega al handler le decisioni di aggiornamento.

UML per la sezione di Alice: gerarchia delle entità e State Pattern dei nemici.



Entità statiche e dinamiche (Entity, StaticEntity, DynamicEntity)

Ho definito la gerarchia di base delle entità di gioco, realizzando **Entity** come classe astratta che raccoglie posizione, dimensioni e stato di attività, con le due specializzazioni

StaticEntity (blocchi, piattaforme) e **DynamicEntity** (player, nemici, power-up, monete in movimento).

All'interno di Entity ho introdotto la classe statica annidata **BoundingBox**, utilizzata per il rilevamento delle collisioni tramite il metodo `overlaps()`. Ho inoltre aggiunto il metodo statico `filterActive()`, che tramite una pipeline Stream restituisce solo le entità ancora attive nel mondo di gioco, eliminando la necessità di iterazioni manuali con controlli inline nel game loop e rendendo il codice più espressivo e dichiarativo.

Entità del mondo (Coin, Block, Flag, Pole)

Ho implementato le entità **Coin e Block**, occupandomi sia della loro logica che del rendering tramite JavaFX. Per Coin ho gestito l'animazione ciclica degli sprite tramite scaling orizzontale controllato dal campo `FRAME_SCALE_X`, che permette di simulare la rotazione della moneta senza utilizzare trasformazioni 3D. Per Block ho definito il comportamento statico di rendering sul canvas, con la distinzione tra i tre tipi disponibili (NORMAL, BRICK, QUESTION) gestita tramite una switch expression.

Ho inoltre implementato Flag e Pole, che segnalano la fine del livello e si integrano con la logica di completamento gestita da **BasicLevel**.

AnimationManager

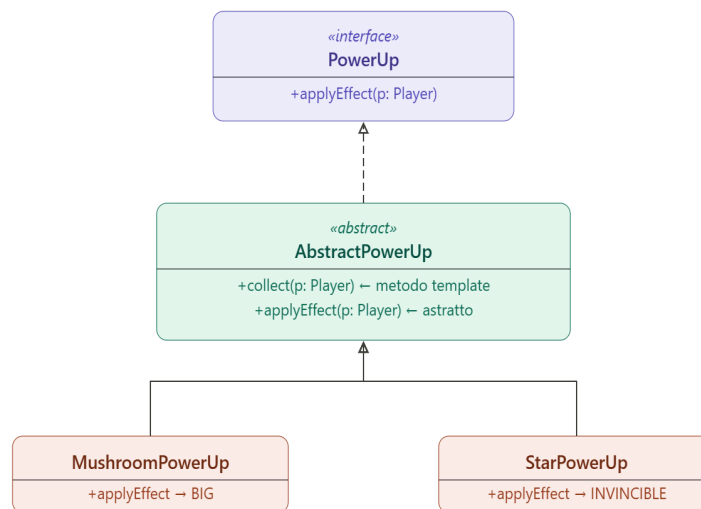
Mi sono occupata anche della gestione delle animazioni tramite la classe **AnimationManager**, responsabile di selezionare e riprodurre il frame corretto per ogni entità in base al suo stato attuale e alla direzione di movimento. Mantiene un registro di sequenze di sprite indicizzate per chiave di animazione. Ad ogni frame di gioco, la classe avanza il contatore interno secondo un timer basato sul delta time, garantendo che la velocità dell'animazione sia indipendente dal frame rate. Il metodo `draw()` applica le trasformazioni necessarie al `GraphicsContext` di JavaFX — in particolare lo scaling negativo sull'asse X con traslazione compensativa per il flip orizzontale degli sprite. Questa soluzione riduce il numero di asset necessari e centralizza tutta la logica di rendering degli sprite in un unico punto, rendendo più semplice aggiungere nuove animazioni. La classe è utilizzata sia dai nemici sia dal player e dai power-up.

Parti in collaborazione

Ho collaborato con Gabriele per integrare correttamente il comportamento fisico all'interno di **DynamicEntity** e per assicurarmi che il tick fisico venisse invocato nel momento corretto all'interno del ciclo `update()` di **EnemyImpl**.

2.2.2 Sabrina — Giocatore e power-up

Gestione dei Power-Up



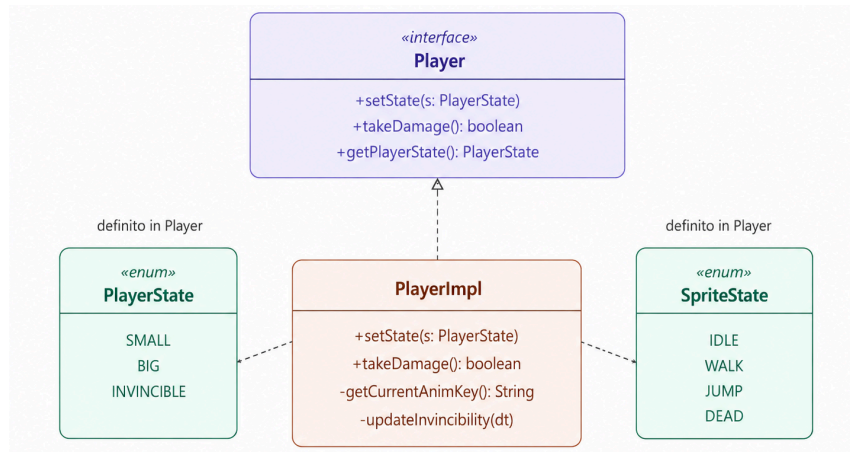
Problema. Nel gioco sono presenti diversi power-up che condividono una parte di comportamento comune, ma producono effetti diversi sul player. Tutti i power-up devono emergere dal blocco, muoversi nel livello, invertire la direzione in caso di collisione e poter essere raccolti dal player. Tuttavia, l'effetto applicato al player cambia in base al tipo di power-up: il fungo trasforma il player nello stato BIG, mentre la stella lo porta nello stato INVINCIBLE. Duplicare questa logica in ogni classe concreta avrebbe reso il sistema fragile e difficile da estendere nel caso di aggiunta futura di nuovi power-up.

Soluzione. La soluzione adottata separa il comportamento comune dall'effetto specifico. L'interfaccia `PowerUp` definisce le caratteristiche generali che ogni power-up deve rispettare, esponendo operazioni come `update()`, `applyEffect(Player)`, `collect(Player)`, `reverseDirection()` e `isEmerging()`. La classe astratta `AbstractPowerUp` implementa la logica condivisa: l'emersione dal blocco, l'attivazione della gravità al termine dell'emersione, l'avvio del movimento orizzontale e l'inversione della direzione. Il metodo `collect(Player)` definisce il flusso comune della raccolta — applica l'effetto al player tramite `applyEffect(Player)`, poi distrugge il power-up — ma lascia il dettaglio dell'effetto alle sottoclassi tramite il metodo astratto `applyEffect(Player)`. `MushroomPowerUp` porta il player allo stato BIG; `StarPowerUp` lo porta allo stato INVINCIBLE e ridefinisce anche il comportamento di aggiornamento per ottenere il rimbalzo quando si trova a terra.

`AbstractPowerUp` definisce la struttura generale dell'operazione di raccolta tramite `collect(Player)` (metodo template), mentre il dettaglio variabile — l'effetto applicato al player — viene delegato alle sottoclassi tramite il metodo astratto `applyEffect(Player)`. Il resto del

sistema può trattare i diversi power-up attraverso il tipo PowerUp senza conoscere le classi concrete, realizzando polimorfismo per sostituzione.

Gestione degli stati del Player

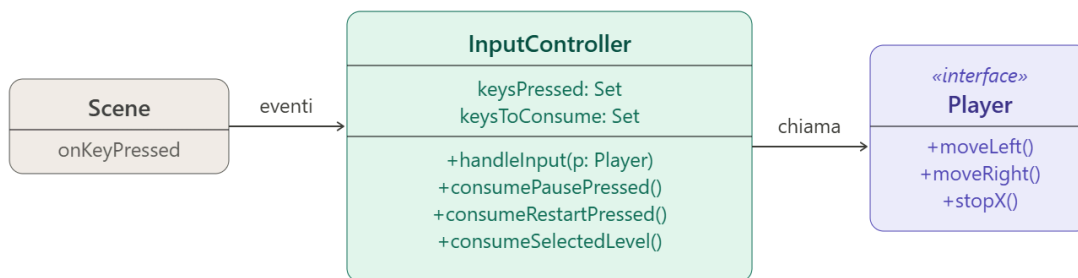


Problema. Il player può assumere stati diversi durante la partita: *SMALL*, *BIG* e *INVINCIBLE*. Ogni stato influenza il comportamento del personaggio — quando è piccolo subisce la morte in caso di danno, quando è grande può tornare piccolo, mentre quando è invincibile resiste ai nemici per un tempo limitato. Oltre allo stato logico, è necessario gestire separatamente lo stato visuale dell'animazione (*IDLE*, *WALK*, *JUMP*, *DEAD*), perché la logica di gioco e la grafica non coincidono sempre: un player può essere logicamente *BIG* ma visualmente trovarsi nello stato *JUMP*.

Soluzione. La gestione del player è stata organizzata separando lo stato logico dallo stato visuale. Nell'interfaccia **Player** sono stati definiti due enum: **PlayerState**, che rappresenta gli stati di gioco, e **SpriteState**, che rappresenta lo stato dell'animazione corrente. Posizionare gli enum nell'interfaccia li rende parte del contratto pubblico, accessibili a tutto il sistema senza dipendere dalla classe concreta **PlayerImpl**. Il metodo `setState()` centralizza le transizioni di stato, gestendo anche gli effetti collaterali come la modifica delle dimensioni del personaggio nel passaggio da *SMALL* a *BIG*. La scelta dell'animazione corrente è delegata al metodo privato `getCurrentAnimKey()`, che seleziona la chiave in base alla combinazione di stato logico e stato visuale tramite una switch expression. La gestione dell'invincibilità è controllata tramite timer nel metodo `updateInvincibility()`: al termine del conto alla rovescia il personaggio torna automaticamente allo stato *BIG*, con effetto visivo di lampeggio.

Gli stati *SMALL*, *BIG* e *INVINCIBLE* modellano esplicitamente le diverse condizioni del personaggio, evitando una gestione basata su variabili booleane separate. Le transizioni tra stati sono centralizzate in `setState()`, che costituisce il punto unico di controllo.

Gestione dell'input da tastiera



Problema. Il gioco deve interpretare i comandi della tastiera e trasformarli in azioni del player. Alcuni comandi, come il movimento, devono rimanere attivi finché il tasto resta premuto. Altri, come pausa, restart o selezione del livello, devono essere eseguiti una sola volta per pressione. Gestire tutti gli input direttamente dentro il player o nel ciclo principale avrebbe creato un forte accoppiamento tra tastiera, logica di controllo e modello.

Soluzione. La gestione degli input è stata isolata nella classe **InputController**. Questa classe registra gli eventi della tastiera provenienti dalla **Scene** e mantiene due insiemi distinti: `keysPressed`, per i tasti attualmente premuti, e `keysToConsume`, per i tasti da consumare una sola volta. Il metodo `handleInput(Player)` traduce lo stato della tastiera in chiamate sul player: se è premuta la freccia sinistra viene chiamato `moveLeft()`, se è premuta la destra `moveRight()`, altrimenti `stopX()`. Il salto è comunicato tramite `setJumpRequested()`. Per i comandi singoli vengono usati metodi dedicati come `consumePausePressed()`, `consumeRestartPressed()` e `consumeSelectedLevel()`, che rimuovono il comando da `keysToConsume` impedendo che la stessa azione venga ripetuta a ogni frame.

Motivazione della scelta. Il player non conosce i tasti della tastiera e non dipende dalla sorgente dell'input: riceve solo comandi astratti come `moveLeft()` e `setJumpRequested()`. Sostituire la tastiera con un'altra sorgente, ad esempio un gamepad, richiederebbe modifiche solo in **InputController**, lasciando invariata la logica del player.

Questa parte è stata successivamente integrata in collaborazione con Lorenzo, soprattutto per collegare **InputController** al **GameManager** e al flusso generale della partita. Sono stati aggiunti comandi consumabili per pausa, restart e selezione del livello dal menu, così da distinguere gli input continui del movimento dagli input che devono essere eseguiti una sola volta per pressione. In questo modo **InputController** non si limita a gestire il movimento del player, ma fornisce anche al **GameManager** le informazioni necessarie per controllare gli stati principali del gioco.

2.2.3 Lorenzo — Livelli, GameManager, HUD e flusso di gioco

BasicLevel come contenitore e gestore del livello

Problema. Il livello deve contenere entità diverse tra loro, come blocchi, monete, nemici, power-up e player, aggiornandole a ogni frame e gestendo le principali interazioni. Una singola classe incaricata di collisioni, nemici, ricompense e completamento del livello sarebbe però risultata troppo estesa e difficile da mantenere.

Soluzione. **BasicLevel** mantiene la lista delle entità e coordina il ciclo di aggiornamento, delegando le responsabilità più specifiche a componenti separati.

LevelCollisionManager gestisce le collisioni con blocchi, piattaforme e bandiera, aggiorna lo stato **onGround**, riconosce i blocchi colpiti dal basso e permette il wall jump.

EnemyInteractionManager gestisce il contatto tra player e nemici, il comportamento dei gusci del Koopa, la sconfitta dei nemici e l'inversione della direzione vicino ai precipizi.

QuestionBlockRewardManager genera monete, funghi e stelle secondo un ordine casuale a rotazione. Le entità create durante l'aggiornamento vengono inserite prima in una lista temporanea e aggiunte al livello alla fine del frame, evitando modifiche concorrenti.

Scelta progettuale. La divisione in manager applica il principio di singola responsabilità: **BasicLevel** rimane il coordinatore generale, mentre ogni componente gestisce una parte precisa del gameplay.

BasicLevelLoader e costruzione dei livelli

Problema. I tre livelli devono avere dimensioni, strutture e difficoltà differenti, includendo piattaforme, precipizi, sezioni verticali, tunnel, blocchi speciali, monete e nemici. Inserire tutta la costruzione dentro **BasicLevelLoader** avrebbe prodotto una classe molto lunga e piena di codice duplicato.

Soluzione. **BasicLevelLoader** ha il solo compito di selezionare il livello richiesto. Ogni livello è descritto da una classe dedicata, rispettivamente **LevelOneDefinition**,

LevelTwoDefinition e **LevelThreeDefinition**, che implementa l'interfaccia

LevelDefinition. La costruzione delle strutture comuni è affidata a **LevelBuilder**, che contiene metodi riutilizzabili per creare terreno, ponti, piramidi, piattaforme sospese, tunnel, sezioni verticali, blocchi interrogativi, monete e gruppi di nemici. In questo modo i dati dei livelli sono separati dalla logica usata per costruirli.

Scelta progettuale. Questa organizzazione riduce la duplicazione e permette di modificare il layout di un livello intervenendo principalmente sulla relativa classe di definizione.

GameManager e flusso della partita

Problema. Il gioco deve coordinare menu, partita in corso, pausa, game over e vittoria. Deve inoltre elaborare gli input, aggiornare il livello, gestire il punteggio, controllare la morte del player e mantenere la camera allineata alla sua posizione.

Soluzione. **GameManager** coordina il flusso principale tramite l'enum *GameState*, che comprende gli stati *MENU*, *PLAYING*, *PAUSED*, *GAME_OVER* e *VICTORY*. Durante la partita richiama **InputController**, aggiorna il livello e comunica allo **ScoreSystem** le monete raccolte e i nemici sconfitti. Gestisce inoltre la perdita delle vite, il caricamento del livello dopo la morte, il completamento tramite la bandiera e il ritorno automatico al menu dopo la vittoria. La camera segue il player senza superare i limiti della mappa e viene aggiornata quando cambiano le dimensioni della finestra.

ScoreSystem

Problema. Punteggio, monete e vite devono essere aggiornati in modo coerente e mostrati immediatamente nell'HUD.

Soluzione. **ScoreSystem** mantiene il punteggio corrente, il numero di monete e le vite disponibili. La partita inizia con tre vite e zero punti. Ogni moneta assegna 100 punti, mentre ogni nemico sconfitto assegna 200 punti. Al raggiungimento di 100 monete viene assegnata una vita extra e il contatore delle monete riparte da zero. La classe gestisce anche la perdita delle vite, il game over e il ripristino dei valori iniziali.

HUD e menu

Problema. Il giocatore deve ricevere informazioni chiare sullo stato della partita e poter scegliere facilmente il livello iniziale.

Soluzione. **HudView** disegna il menu, l'HUD e i messaggi relativi agli stati della partita. Durante il gameplay mostra punteggio, monete, vite e livello corrente; nel menu permette di selezionare uno dei tre livelli; durante pausa, game over e vittoria mostra i relativi messaggi. **LevelBackgroundRenderer** si occupa invece dello sfondo dei livelli, disegnando cielo, sole, nuvole, colline e terreno con colori differenti in base al livello. Il canvas segue le dimensioni della finestra, consentendo alla rappresentazione grafica di adattarsi al ridimensionamento.

2.2.4 Gabriele – Sistema fisico

BasicPhysics, CollisionDetector, CollisionResolver

Problema. Le entità dinamiche devono essere soggette alla gravità, aggiornare la loro posizione ad ogni frame e non compenetrare le entità statiche. Concentrare tutta questa logica in *DynamicEntity* avrebbe accoppiato la gerarchia delle entità all'algoritmo fisico, rendendo impossibile testarlo in isolamento o sostituirlo.

Soluzione. Il sistema fisico è articolato in tre interfacce con responsabilità distinte:

-BasicPhysics : applica la gravità e integra la velocità in posizione sia verticale che orizzontale

-CollisionDetector (rileva se due *GameObject* (siano essi statici o dinamici) si sovrappongono e determina il lato di collisione

-CollisionResolver: risolve la collisione spostando l'oggetto dinamico in base a dove essa è avvenuta.

DynamicEntity riceve *BasicPhysics* nel costruttore; *BasicLevel* riceve *CollisionDetector* e *CollisionResolver* per composizione. Le implementazioni concrete (*BasicPhysicsImpl*, *CollisionDetectorImpl*, *CollisionResolverImpl*) sono invisibili al resto del sistema.

CollisionResult è un record che trasporta i due oggetti coinvolti e il lato della collisione (*CollisionSide*): separare il rilevamento dalla risoluzione permette a *BasicLevel* di raccogliere tutti i risultati prima di risolverli, aprendo la strada a ottimizzazioni future (es. prioritizzare le collisioni verticali).

Pattern applicato: Strategy. Le tre interfacce fisiche sono strategie intercambiabili: sostituire *BasicPhysicsImpl* con una fisica più avanzata (es. con attrito o rimbalzo) non richiede modifiche a *DynamicEntity* né a *BasicLevel*.

GameObject: entità che interagisce con la fisica di base e sta alla base di tutte gli oggetti di gioco.

Capitolo 3 - Sviluppo

3.1 Testing automatizzato

Il progetto include una suite di test automatizzati sviluppata con **JUnit 5**, con l'obiettivo di verificare la correttezza della logica di gioco e ridurre il rischio di regressioni a seguito di modifiche al codice. Poiché il platformer presenta numerose interazioni tra entità, fisica, input, livelli e gestione dello stato della partita, i test sono stati organizzati in modo da controllare sia il comportamento delle singole classi sia alcune integrazioni significative tra i componenti principali.

La maggior parte dei test si concentra sul **model**, cioè sulla parte logica dell'applicazione, indipendente dalla view. Questo ha permesso di testare automaticamente i comportamenti fondamentali del gioco senza richiedere l'intervento dell'utente. Sono stati quindi verificati i movimenti delle entità, le collisioni, la raccolta degli oggetti, l'applicazione dei power-up, la morte o sconfitta delle entità e l'aggiornamento del punteggio. L'interfaccia grafica non è stata oggetto principale dell'apparato di test, mentre la logica sottostante alla partita è stata testata in modo automatico.

Un primo gruppo di test riguarda il sistema fisico del gioco. In particolare, sono stati verificati il corretto funzionamento dei vettori, le operazioni di somma, sottrazione, scalatura e clonazione, oltre alla gestione degli oggetti fisici e al rilevamento delle collisioni. Questi test sono importanti perché il movimento delle entità e la risoluzione delle interazioni con il mondo di gioco dipendono direttamente dal componente fisico.

Sono stati poi realizzati test specifici per il **Player**, controllando il movimento orizzontale, il salto, l'impossibilità di saltare quando il personaggio non si trova a terra, l'arresto del movimento sull'asse X e le transizioni di stato dovute a danni o power-up. I test verificano, ad esempio, il passaggio da stato normale a stato grande, l'attivazione dell'invincibilità, la fine dell'invincibilità dopo il tempo previsto, la morte del giocatore quando viene colpito in stato piccolo e il ritorno allo stato piccolo quando viene danneggiato da grande.

Un'attenzione particolare è stata dedicata anche ai nemici, in particolare **Goomba** e **Koopa**. Per il Goomba sono stati controllati lo stato iniziale, la posizione, le dimensioni, la possibilità di colpire il player e il comportamento in seguito allo schiacciamento. I test verificano inoltre che, dopo essere stato sconfitto, il nemico non sia più pericoloso e venga eliminato dopo il tempo previsto. Per il Koopa sono stati testati più stati, tra cui camminata, guscio fermo e

guscio in movimento. Sono stati verificati il cambio di stato dopo lo stomp, la direzione del guscio dopo il calcio, la velocità sull'asse X, la capacità del guscio in movimento di sconfiggere altri nemici e il fatto che alcune azioni non producano effetti indesiderati quando il Koopa si trova già in uno stato specifico.

La suite comprende anche test per gli oggetti del mondo di gioco, come **Coin**, **Block**, **Flag** e **Pole**. Per le monete sono stati testati sia lo stato statico sia il comportamento delle monete generate dai blocchi, verificando il movimento verso l'alto e la successiva rimozione. Per i blocchi sono stati controllati i diversi tipi, in particolare i blocchi normali, i brick block e i blocchi QUESTION, verificando che questi ultimi possano generare contenuto solo al primo colpo. Per l'asta e la bandiera di fine livello sono stati testati la posizione iniziale, le dimensioni, l'avvio dell'abbassamento della bandiera e il suo arresto una volta raggiunta la posizione finale.

I power-up sono stati testati separatamente per verificare sia il comportamento generale sia gli effetti specifici sul player. I test controllano l'emersione dal blocco, l'inizio del movimento orizzontale, l'inversione della direzione, la raccolta e la distruzione dell'oggetto dopo l'applicazione dell'effetto. In particolare, il **MushroomPowerUp** viene verificato per il passaggio del player allo stato grande, mentre lo **StarPowerUp** viene testato per l'attivazione dello stato di invincibilità.

Una parte significativa del testing riguarda la gestione dei livelli tramite **BasicLevel**. I test verificano la raccolta delle monete quando il bounding box del player si sovrappone a quello della moneta, l'assenza di raccolta quando non vi è collisione, il reset del conteggio delle monete raccolte dopo l'aggiornamento del punteggio e la corretta sostituzione del player all'interno del livello. Sono inoltre controllati comportamenti più legati al gameplay, come il movimento orizzontale del player a terra, le collisioni laterali con i blocchi, il wall jump, la generazione di monete, funghi e stelle dai blocchi QUESTION, l'interazione con il palo di fine livello e la sconfitta dei nemici tramite invincibilità o guscio del Koopa.

Il caricamento e la costruzione dei livelli sono stati verificati attraverso test dedicati a **BasicLevelLoader**, **LevelBuilder** e alle classi di definizione dei livelli. I test controllano che ogni livello venga creato con i metadati corretti, che il player sia posizionato nello spawn previsto e che un numero di livello non riconosciuto produca il caricamento del primo livello. Vengono inoltre verificate la presenza di layout differenti, del palo e della bandiera di completamento e la corretta costruzione delle strutture comuni.

Sono stati aggiunti controlli sulla validità dei layout: i blocchi solidi non devono sovrapporsi, le monete non devono trovarsi all'interno dei blocchi, i blocchi QUESTION devono essere

raggiungibili con un salto normale e le piattaforme devono lasciare spazio sufficiente al player nello stato BIG. I test verificano inoltre che i nemici inizino su una superficie valida e dispongano di spazio sufficiente per muoversi. Per il terzo livello viene controllata la presenza di piattaforme attraversabili in tutti i precipizi.

Le responsabilità estratte da **BasicLevel** sono state verificate anche in isolamento. I test di **LevelCollisionManager** controllano collisioni verticali e laterali, aggiornamento dello stato **onGround**, blocchi colpiti dal basso e wall jump. I test di **EnemyInteractionManager** verificano le interazioni tra player, nemici e gusci del Koopa, mentre quelli di **QuestionBlockRewardManager** controllano che ogni ciclo di ricompense contenga monete, un fungo e una stella in ordine casuale.

Lo **ScoreSystem** è stato testato separatamente verificando i valori iniziali di punteggio, monete e vite, l'assegnazione dei punti, la perdita delle vite, il game over e il reset della partita. È stato inoltre controllato che ogni moneta assegni 100 punti, che ogni nemico sconfitto ne assegni 200 e che ogni gruppo di cento monete produca una nuova vita, anche quando la soglia viene raggiunta più volte.

Sono stati aggiunti anche test per il **GameManager**, responsabile della gestione generale della partita. In questo caso sono state verificate le principali transizioni di stato dell'applicazione, tra cui menu, gioco, pausa, vittoria e ritorno al menu. I test controllano l'avvio della partita, la pausa e la ripresa, il ritorno al menu dopo la vittoria e la successiva selezione di un livello, il reset del punteggio quando viene iniziata una nuova partita.

Il componente **InputController** è stato testato per assicurare una corretta gestione dei comandi da tastiera. I test distinguono tra comandi continui, come il movimento laterale, e comandi impulsivi, come pausa, salto, ritorno al menu, restart e selezione del livello. In particolare, viene verificato che i comandi impulsivi vengano consumati una sola volta per ogni pressione del tasto, evitando che una singola pressione produca più azioni consecutive non desiderate.

Nel complesso, l'approccio adottato ha permesso di coprire automaticamente le funzionalità più importanti del progetto: fisica, movimento, collisioni, stati del player, comportamento dei nemici, power-up, oggetti del mondo, caricamento dei livelli, gestione degli input, punteggio e stati principali della partita. La presenza di test automatici eseguibili tramite JUnit 5 contribuisce a garantire maggiore affidabilità durante l'evoluzione del codice e permette di individuare rapidamente eventuali regressioni nelle meccaniche fondamentali del gioco.

3.2 Note di sviluppo

3.2.1 Alice Franzolini

Feature avanzate del linguaggio utilizzate

- **Utilizzo di List, Stream e method reference per il filtraggio delle entità attive**
Classe: `AbstractEntity`
Permalink:
<https://github.com/alicefranzolini/OOP25-platformer/blob/d1bc8b58fbcbb6af8ec8ec0226c8b8edc574a40/src/main/java/it/unibo/platformer/model/entities/AbstractEntity.java#L163>
- **Utilizzo di Optional per la gestione dello stato del nemico**
Classe: `AbstractEnemyImpl`
Permalink:
<https://github.com/alicefranzolini/OOP25-platformer/blob/d1bc8b58fbcbb6af8ec8ec0226c8b8edc574a40/src/main/java/it/unibo/platformer/model/entities/enemies/AbstractEnemyImpl.java#L25>
- **Utilizzo di interfaccia marker per identificare stati di camminata**
Classe: `AbstractEnemyImpl`
Permalink:
<https://github.com/alicefranzolini/OOP25-platformer/blob/d1bc8b58fbcbb6af8ec8ec0226c8b8edc574a40/src/main/java/it/unibo/platformer/model/entities/enemies/AbstractEnemyImpl.java#L156>
- **Utilizzo di lambda e map() su Optional per la delega ai gestori di stato**
Classe: `AbstractEnemyImpl`
Permalink:
<https://github.com/alicefranzolini/OOP25-platformer/blob/d1bc8b58fbcbb6af8ec8ec0226c8b8edc574a40/src/main/java/it/unibo/platformer/model/entities/enemies/AbstractEnemyImpl.java#L124>
- **Utilizzo di factory method per la creazione di entità con stato iniziale differente**
Classe: `Coin`
Permalink:
<https://github.com/alicefranzolini/OOP25-platformer/blob/d1bc8b58fbcbb6af8ec8ec0226c8b8edc574a40/src/main/java/it/unibo/platformer/model/entities/world/Coin.java#L56>
- **Utilizzo di switch expression con pattern arrow**
Classe: `Block`
Permalink:
<https://github.com/alicefranzolini/OOP25-platformer/blob/d1bc8b58fbcbb6af8ec8ec0226c8b8edc574a40/src/main/java/it/unibo/platformer/model/entities/world/Block.java#L68>
- **Utilizzo di trasformazioni del GraphicsContext per il flip orizzontale degli sprite**
Classe: `AnimationManager`

Permalink:

<https://github.com/alicefranzolini/OOP25-platformer/blob/d1bc8b58fcbbe6af8ec8ec0226c8b8edc574a40/src/main/java/it/unibo/platformer/view/AnimationManager.java#L87>

3.2.2 Gabriele Fortarezza

Feature avanzate del linguaggio e dell'ecosistema Java utilizzate:

- **Utilizzo di enum per la gestione dei lati di collisione:**

Class: CollisionSide

Permalink:

<https://github.com/alicefranzolini/OOP25-platformer/blame/f6fde177ac087a2c8848ab4a322e93bb174e037f/src/main/java/it/unibo/platformer/model/physics/impl/CollisionSide.java#L6>

3.2.3 Sabrina Magi

Feature avanzate del linguaggio e dell'ecosistema Java utilizzate

- **Utilizzo di switch expression**

Classe: PlayerImpl

Permalink:

<https://github.com/alicefranzolini/OOP25-platformer/blob/b6ffa3bd9e0eb4b6a4c9cab2a8564c5f52f79611/src/main/java/it/unibo/platformer/model/entities/players/PlayerImpl.java#L130>

- **Uso di lambda expressions**

Classe: InputController

Permalink:

<https://github.com/alicefranzolini/OOP25-platformer/blob/c9a73ce3ca50f8255b23cfd9e270d8fbaf73b749/src/main/java/it/unibo/platformer/controller/InputController.java#L33>

- **Uso di JavaFX per gestione grafica e rendering su canvas**

Classe: PlayerImpl

Permalink:

<https://github.com/alicefranzolini/OOP25-platformer/blob/b6ffa3bd9e0eb4b6a4c9cab2a8564c5f52f79611/src/main/java/it/unibo/platformer/model/entities/players/PlayerImpl.java#L222>

- **Uso di JavaFX per rendering dei power-up**

Classe: MushroomPowerUp

Permalink:

<https://github.com/alicefranzolini/OOP25-platformer/blob/c9a73ce3ca50f8255b23cfd9e270d8fbaf73b749/src/main/java/it/unibo/platformer/model/entities/powerup/MushroomPowerUp.java#L55>

- **Uso di JavaFX rendering grafico personalizzato**

Classe: StarPowerUp

Permalink:

<https://github.com/alicefranzolini/OOP25-platformer/blob/c9a73ce3ca50f8255b23cfd9e270d8fbaf73b749/src/main/java/it/unibo/platformer/model/entities/powerup/StarPowerUp.java#L86>

- **Uso di Math e array per rendering personalizzato della stella**

Classe: StarPowerUp

Permalink:

<https://github.com/alicefranzolini/OOP25-platformer/blob/c9a73ce3ca50f8255b23cfd9e270d8fbaf73b749/src/main/java/it/unibo/platformer/model/entities/powerup/StarPowerUp.java#L93>

3.2.4 Lorenzo Vasini

Feature avanzate del linguaggio e dell'ecosistema Java utilizzate

- **Utilizzo di Map e HashMap per memorizzare le posizioni precedenti delle entità dinamiche durante la risoluzione delle collisioni**

Classe: BasicLevel

Permalink:

<https://github.com/alicefranzolini/OOP25-platformer/blob/f6fde177aco87a2c8848ab4a322e93bb174e037f/src/main/java/it/unibo/platformer/model/level/BasicLevel.java#L176-L181>

- **Uso di lambda expressions con removeif() per eliminare le entità non più attive**

Classe: BasicLevel

Permalink:

<https://github.com/alicefranzolini/OOP25-platformer/blob/f6fde177aco87a2c8848ab4a322e93bb174e037f/src/main/java/it/unibo/platformer/model/level/BasicLevel.java#L224-L228>

- **Utilizzo di interfacce e polimorfismo per separare la definizione dei livelli dalla loro costruzione**

Classe: LevelDefinition, BasicLevelLoader

Permalink:

<https://github.com/alicefranzolini/OOP25-platformer/blob/f6fde177aco87a2c8848ab4a322e93bb174e037f/src/main/java/it/unibo/platformer/model/level/LevelDefinition.java#L6-L28>

<https://github.com/alicefranzolini/OOP25-platformer/blob/f6fde177aco87a2c8848ab4a322e93bb174e037f/src/main/java/it/unibo/platformer/model/level/BasicLevelLoader.java#L19-L34>

- **Utilizzo di enum e Collections.shuffle() per la generazione casuale a rotazione delle ricompense**
 Classe: QuestionBlockRewardManager
 Permalink:
<https://github.com/alicefranzolini/OOP25-platformer/blob/f6fde177ac087a2c8848ab4a322e93bb174e037f/src/main/java/it/unibo/platformer/model/level/QuestionBlockRewardManager.java#L49-L67>
- **Utilizzo di enum per rappresentare gli stati principali della partita**
 Classe: GameManager
 Permalink:
<https://github.com/alicefranzolini/OOP25-platformer/blob/f6fde177ac087a2c8848ab4a322e93bb174e037f/src/main/java/it/unibo/platformer/controller/GameManager.java#L28-L39>
- **Utilizzo del property binding di JavaFX per adattare il canvas alle dimensioni della finestra**
 Classe: Main
 Permalink:
<https://github.com/alicefranzolini/OOP25-platformer/blob/f6fde177ac087a2c8848ab4a322e93bb174e037f/src/main/java/it/unibo/platformer/Main.java#L33-L36>
- **Utilizzo di una classe anonima basata su AnimationTimer per realizzare il game loop**
 Classe: Main
 Permalink:
<https://github.com/alicefranzolini/OOP25-platformer/blob/f6fde177ac087a2c8848ab4a322e93bb174e037f/src/main/java/it/unibo/platformer/Main.java#L46-L63>

Capitolo 4 - Commenti finali

4.1 Autovalutazione e lavori futuri

4.1.1 Alice Franzolini

Sono complessivamente soddisfatta del lavoro svolto. Mi sono occupata della modellazione dei nemici e della gerarchia delle entità, un compito che ho trovato stimolante soprattutto nella fase di progettazione della classe AbstractEnemyImpl e nella scelta di come gestire gli stati interni in modo pulito ed espressivo. Ritengo che il codice prodotto sia facilmente estendibile, ad esempio per aggiungere nuovi tipi di nemici senza dover modificare la logica esistente.

Il mio ruolo all'interno del gruppo è stato prevalentemente collaborativo, l'unica cosa che cambierei riguarda l'organizzazione temporale del gruppo: avremmo potuto iniziare prima alcune parti del progetto, il che avrebbe lasciato più tempo per rifinire i dettagli e affrontare con più calma le fasi finali. Nonostante questo, il risultato finale mi soddisfa.

Non escluderei di portare avanti il progetto in futuro, ad esempio aggiungendo nuovi tipi di nemici, ulteriori power-up o livelli più articolati, dato che l'architettura realizzata si presta bene a questo tipo di estensioni.

4.1.2 Gabriele Fortarezza

Sono abbastanza soddisfatto del lavoro svolto. Mi sono occupato della gestione della fisica di base con la classe BasicPhysics, delle collisioni con CollisionDetector, CollisionResolver, ossia quei componenti che permettono agli oggetti nel gioco di interagire tra loro in maniera realistica.

Sono soddisfatto di come ho gestito la fisica utilizzando i vettori per lo spostamento e la velocità, capendo prima il meccanismo e riuscendo anche ad incastrare all'interno del progetto altre materie del corso.

Un punto negativo è stato invece secondo me il fatto che dividendo il lavoro del gruppo in maniera netta perché in alcuni momenti poteva rallentare il lavoro, invece con una divisione più dinamica si sarebbe potuto lavorare in maniera migliore.

Il progetto in futuro per quanto mi riguarda potrebbe implementare la fisica in altri elementi, come per esempio spazio, acqua... oppure implementando nuovi livelli con della fisica particolare per rendere il gioco più divertente.

4.1.3 Sabrina Magi

Sono complessivamente soddisfatta del lavoro svolto. Mi sono occupata principalmente della gestione del Player, dei PowerUp e dell'InputController, cioè di componenti centrali per il gameplay. In particolare, ho lavorato sulla logica degli stati del personaggio, sulla raccolta dei potenziamenti e sulla traduzione degli input in azioni di gioco.

Un punto di forza del mio lavoro è stata la separazione tra responsabilità diverse. La logica del player è stata distinta dalla gestione degli input, e la struttura dei power-up permette di condividere il comportamento comune lasciando alle sottoclassi solo l'effetto specifico. Questo rende il codice più ordinato e più semplice da estendere.

Un aspetto che migliorerei riguarda invece l'organizzazione del lavoro nel gruppo. A posteriori penso che forse abbiamo diviso le parti in modo troppo netto, assegnando a ciascuno componenti molto specifiche. Questa scelta ha facilitato il lavoro individuale, ma in alcuni momenti ha reso più difficile avere una visione completa del progetto e integrare bene le varie parti.

Secondo me sarebbe stato meglio distribuire il lavoro in modo più trasversale, facendo sì che ogni membro contribuisse almeno in parte al model e anche ad altre componenti (view e controller). In questo modo tutti avremmo avuto una comprensione più completa dell'architettura generale e sarebbe stato più semplice mantenere coerenza tra le diverse sezioni del codice.

In futuro il progetto potrebbe essere ampliato aggiungendo nuovi power-up, nuovi stati del player o nuove modalità di controllo.

4.1.4 Lorenzo Vasini

Sono soddisfatto del lavoro svolto, soprattutto perché la mia parte ha riguardato l'integrazione di diversi componenti del gioco. Mi sono occupato principalmente della struttura e della costruzione dei livelli, del GameManager, dell'HUD, dello ScoreSystem e della gestione generale del flusso della partita. Ho inoltre collaborato con Sabrina sull'InputController, partendo dalla struttura iniziale realizzata da lei e integrandola con i comandi necessari per menu, pausa, ripresa, riavvio e selezione dei livelli.

La parte più interessante è stata la costruzione dei livelli, perché ha richiesto di bilanciare piattaforme, nemici, monete, blocchi speciali, precipizi e sezioni verticali. Durante lo sviluppo la costruzione è stata separata in BasicLevelLoader, nelle classi LevelOneDefinition, LevelTwoDefinition e LevelThreeDefinition e nel componente comune LevelBuilder. Questa suddivisione ha permesso di ridurre la duplicazione e di rendere più leggibili e modificabili i layout, comprendenti scalate, tunnel, piramidi, piattaforme sospese e sezioni dedicate al wall jump.

Un altro aspetto importante è stato il GameManager, che collega input, livello corrente, ScoreSystem, camera e stati della partita. Attraverso l'enum GameState coordina menu, gameplay, pausa, game over e vittoria. Durante il lavoro ho cercato di mantenere questa classe come coordinatore generale, spostando invece le interazioni specifiche del livello in componenti dedicati come LevelCollisionManager, EnemyInteractionManager e QuestionBlockRewardManager.

In futuro migliorerei ulteriormente la separazione tra logica e grafica, soprattutto spostando ancora più responsabilità di rendering fuori dal model. Aggiungerei anche livelli caricati da file esterni, così da poter modificare le mappe senza intervenire direttamente sul codice.

Una difficoltà importante è stata l'integrazione tra parti sviluppate da membri diversi del gruppo. Alcune classi, come quelle relative alle entità dinamiche, alla fisica, ai livelli e al player, sono molto collegate tra loro: una modifica a una di queste parti può influenzare collisioni, movimento, power-up e test. Per questo motivo è stato necessario fare diversi merge e riallineamenti, soprattutto dopo l'introduzione delle regole di checkstyle e dei refactor sui nomi delle classi base.

Un'altra difficoltà è stata mantenere coerente il comportamento del gameplay durante l'integrazione. Ad esempio, la gestione dello stato onGround, delle collisioni con i blocchi, dei nemici e dei power-up richiede che fisica, player, entità e livelli siano allineati. Quando una parte cambia interfaccia o comportamento, anche le altre devono adattarsi correttamente. Questo ha reso evidente l'importanza di definire contratti chiari tra i componenti e di verificare spesso il progetto con test e compilazione.

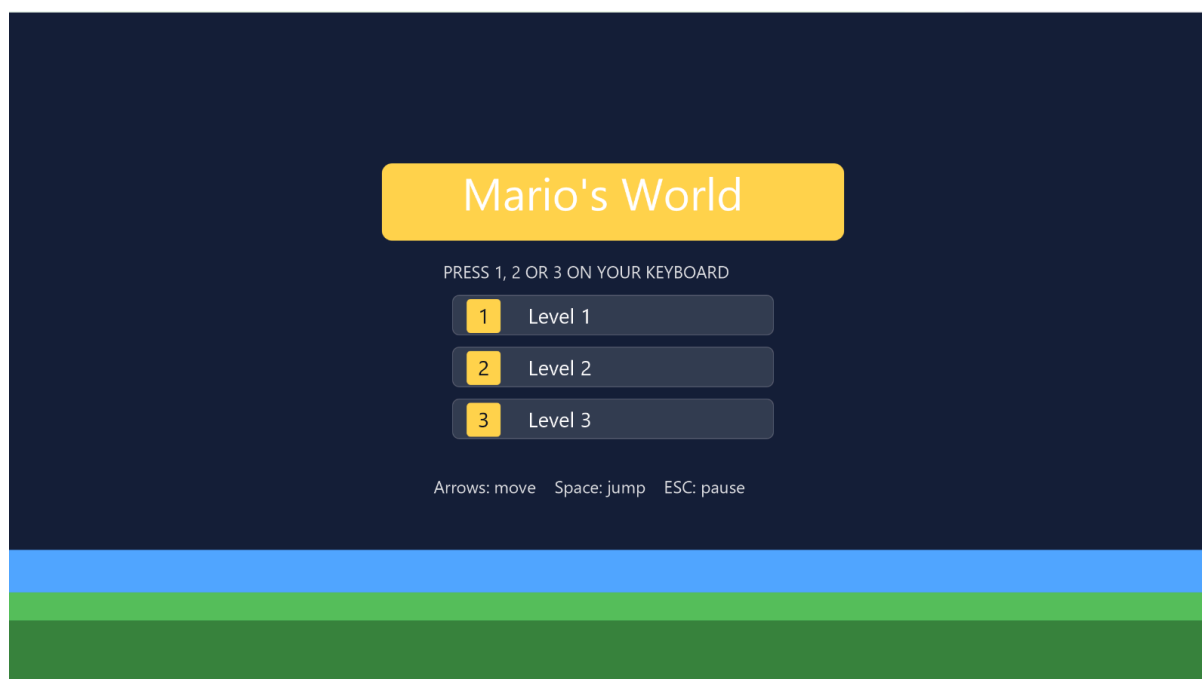
GUIDA UTENTE

1. Introduzione

Il presente capitolo descrive le modalità di installazione, avvio e utilizzo di OOP25-Platformer, un videogioco a piattaforme 2D a scorrimento laterale sviluppato in Java con l'ausilio del framework grafico JavaFX. Obiettivo del giocatore è guidare il proprio personaggio attraverso i livelli disponibili, superando ostacoli, sconfiggendo i nemici e raccogliendo monete, fino al raggiungimento della bandiera di fine livello.

2. Menu principale

All'avvio dell'applicazione viene mostrata una schermata di menu, dalla quale è possibile selezionare il livello da giocare tra i tre disponibili, tramite gli appositi tasti numerici (si veda la sezione successiva).



3. Comandi di gioco

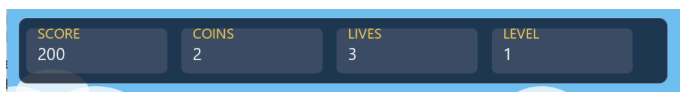
Il gioco è controllato interamente tramite tastiera. Di seguito è riportato l'elenco completo dei comandi disponibili:

Tasto	Azione
Freccia sinistra	Sposta il personaggio verso sinistra
Freccia destra	Sposta il personaggio verso destra
Barra spaziatrice	Esegue il salto

Esc	Mette in pausa / riprende la partita
M	Torna al menu principale
R	Riavvia il livello corrente
1,2 o 3	Avvia / seleziona il Livello 1 2 o 3

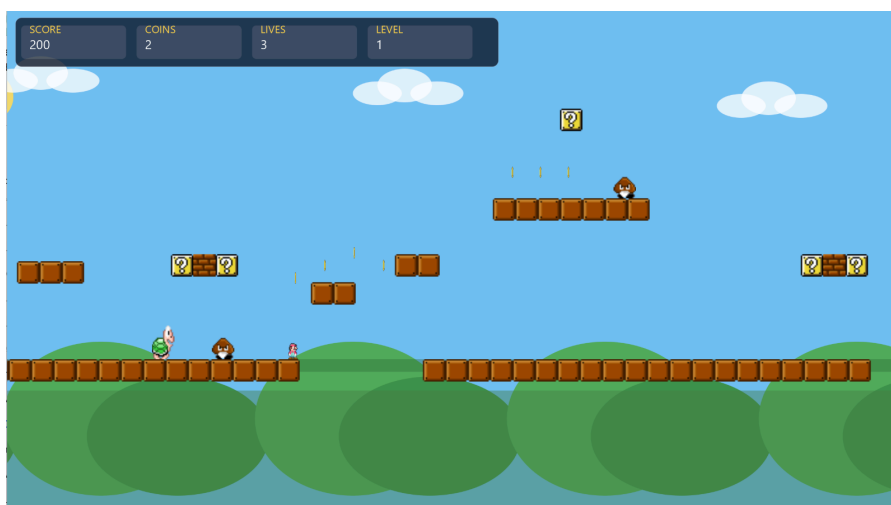
4. Interfaccia di gioco (HUD)

Durante la partita, a schermo è costantemente visibile un'interfaccia (HUD, *Heads-Up Display*) che mostra in tempo reale le informazioni principali relative allo stato della partita, quali il punteggio corrente, il numero di vite residue e il numero di monete raccolte.



5. Meccaniche di gioco

- Movimento e salto: il personaggio può muoversi lateralmente ed effettuare salti per superare ostacoli, dislivelli e nemici.
- Vite e punteggio: il giocatore dispone di un numero limitato di vite; la raccolta delle monete incrementa il punteggio e, al raggiungimento di determinate soglie, garantisce una vita bonus. Il gioco termina (Game Over) quando le vite a disposizione si esauriscono.
- Nemici: lungo i livelli sono presenti nemici di diverso tipo, sconfiggibili saltandovi sopra; alcuni nemici, una volta colpiti, possono essere ulteriormente utilizzati come proiettili contro altri avversari.
- Blocchi interattivi: alcuni blocchi presenti nei livelli, se colpiti dal basso, rilasciano ricompense (ad esempio monete aggiuntive).
- Completamento del livello: il livello si considera concluso quando il personaggio raggiunge il pennone con la bandiera posto al termine del percorso.



6. Livelli disponibili

L'applicazione mette a disposizione tre livelli di gioco, ciascuno caratterizzato da una diversa disposizione di piattaforme, ostacoli e nemici, selezionabili singolarmente dal menu principale.

7. Stati dell'applicazione

Il gioco gestisce i seguenti stati:

- Menu: schermata iniziale di selezione del livello.
- In gioco (Playing): stato attivo durante la partita.
- Pausa (Paused): stato raggiungibile durante la partita tramite il tasto Esc, che sospende temporaneamente l'esecuzione.
- Game Over: raggiunto in seguito all'esaurimento delle vite disponibili.
- Vittoria (Victory): raggiunto al completamento con successo di un livello.